

Moteur Physique

Eliott ROSE - Corentin BOIS COTTE

Septembre – Octobre 2025

Table des matières

1	Introduction	3
2	Broad Phase	4
2.1	Principe et structure de l'arbre	4
2.2	Détection des paires potentielles	5
2.3	Résultats	5
3	Narrow Phase	8
3.1	Algorithme GJK	8
3.2	Algorithme EPA	9
4	Corps rigides personnalisés (CustomRigidbody)	10
4.1	Résistance aérodynamique	10
5	Réponse aux collisions	11
5.1	Formulation du contact	11
5.2	Impulsion normale (collision élastique/inélastique)	11
5.3	Friction tangentielle	11
5.4	Résistance au roulement et correction	12
5.5	Résultats	12
6	Démonstration	14
7	Architecture du projet	15
8	Conclusion	16

1 Introduction

Dans le cadre de ce projet, nous avons réalisé et implémenté un moteur physique 3D personnalisé, avec pour objectif de comprendre en profondeur les principes fondamentaux régissant la détection et la réponse aux collisions dans un environnement interactif. Le moteur a été développé en *C#* et intégré dans l'environnement *Unity* uniquement pour la partie rendu graphique, tandis que nous avons réalisé l'ensemble des calculs physiques manuellement.

Notre moteur repose sur une architecture complète de simulation, comprenant la gestion des corps rigides (*Rigid Bodies*) ainsi que la création de colliders personnalisés, tels que des boîtes (*Box Colliders*) et des sphères (*Sphere Colliders*). L'objectif principal était de reproduire un comportement physique réaliste en contrôlant chaque étape du pipeline de détection et de résolution des collisions.

La phase de détection des collisions est divisée en deux grandes étapes :

- La **Broad Phase**, où nous utilisons des volumes englobants de type AABB (*Axis-Aligned Bounding Boxes*) organisés dans une structure hiérarchique sous la forme d'un arbre binaire (*AABB Binary Tree*), permettant de réduire efficacement le nombre de paires d'objets à tester.
- La **Narrow Phase**, où la détection précise est assurée par l'algorithme GJK (*Gilbert-Johnson-Keerthi*), complété par l'algorithme EPA (*Expanding Polytope Algorithm*) afin d'obtenir les informations de pénétration nécessaires au calcul des réponses aux collisions.

Enfin, pour la **réponse aux collisions**, nous avons implémenté un modèle de résolution linéaire et angulaire intégrant à la fois la friction statique et la friction dynamique, afin de reproduire un comportement plus crédible lors des interactions entre objets.

Ce projet nous a permis d'approfondir notre compréhension des principes physiques, des structures de données et des algorithmes de géométrie computationnelle, tout en développant un outil concret illustrant les fondements d'un moteur physique moderne.

2 Broad Phase

La **Broad Phase** a pour objectif de réduire le nombre de paires d'objets à tester pour les collisions. En effet, le test exhaustif entre toutes les combinaisons d'objets présente une complexité en $O(n^2)$, ce qui devient rapidement prohibitif. Pour pallier ce problème, nous avons implémenté une structure hiérarchique sous forme d'un **AABB Binary Tree**.

2.1 Principe et structure de l'arbre

Chaque *collider* présent dans la scène est associé à une **AABB** (*Axis-Aligned Bounding Box*) qui englobe sa géométrie. Ces boîtes sont mises à jour à chaque frame en fonction de la position et de la rotation des objets.

Les AABB sont ensuite insérées dans un arbre binaire, où chaque nœud interne représente une *zone englobante abstraite*, tandis que les feuilles représentent les colliders réels. L'insertion se fait de manière hiérarchique en comparant les coûts d'union entre les volumes existants :

```
function InsertAABB(bound, collider):
    if tree is empty:
        root = createNode(bound, collider)
        return

    current = root
    while not current.isLeaf:
        leftCost = UnionCost(current.left.AABB, bound)
        rightCost = UnionCost(current.right.AABB, bound)
        current = (leftCost < rightCost) ? current.left : current.right

    parent = createParent(current, bound)
    updateTreeHierarchy(parent)
```

Ainsi, chaque insertion fusionne deux volumes englobants dans un nœud parent, de sorte que la structure reste équilibrée et spatialement cohérente.

2.2 Détection des paires potentielles

Une fois l'arbre construit, la détection consiste à parcourir récursivement les nœuds et à vérifier les intersections entre AABB. Seules les paires de feuilles dont les volumes se superposent sont ajoutées à la liste des paires potentielles :

```
function DetectAllCollisions(node):  
    if node.isLeaf: return  
    checkOverlap(node.left, node.right)  
    DetectAllCollisions(node.left)  
    DetectAllCollisions(node.right)
```

Cette approche permet de réduire drastiquement le nombre de tests nécessaires à la **Narrow Phase**, tout en conservant une structure dynamique et adaptable.

2.3 Résultats

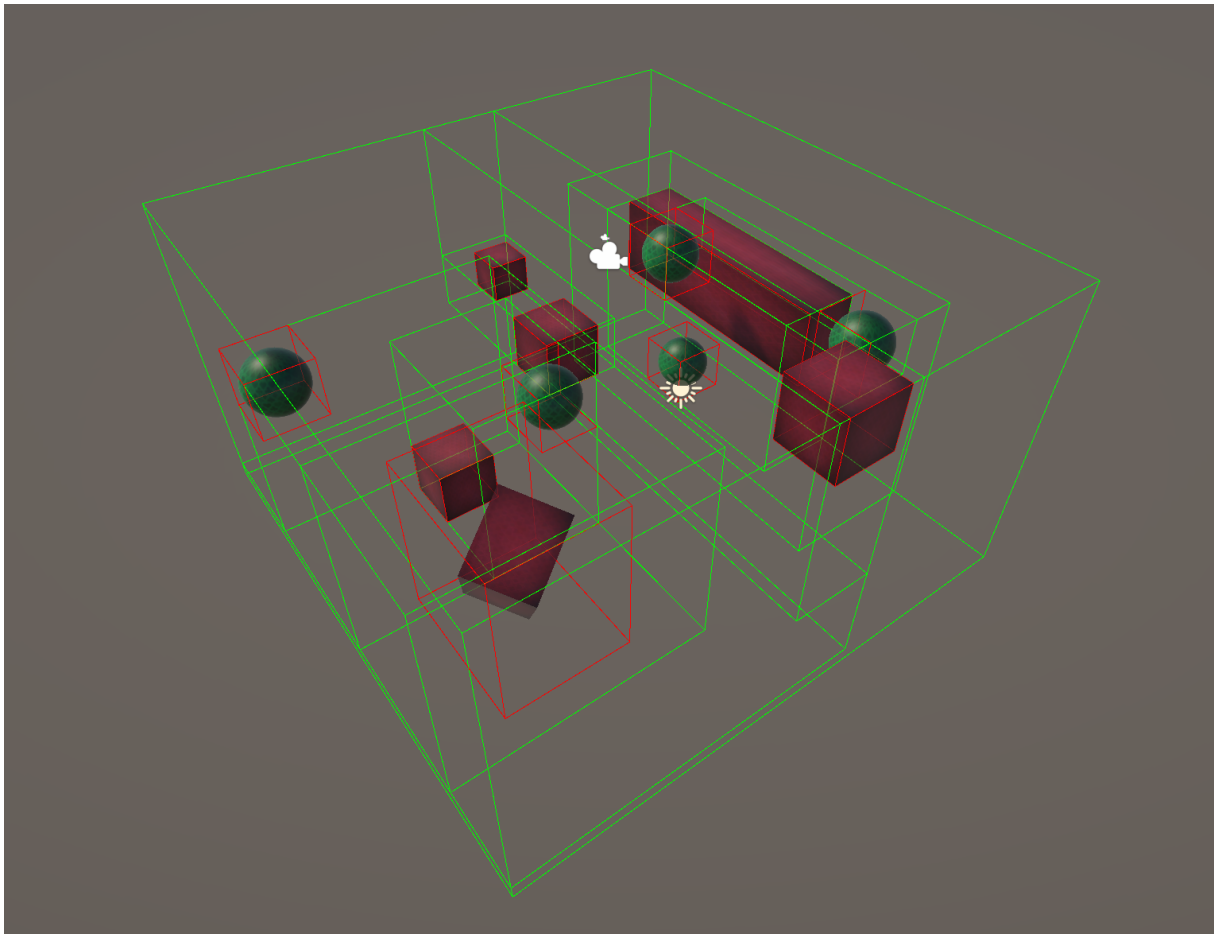


FIGURE 1 – AABB Binary Tree.

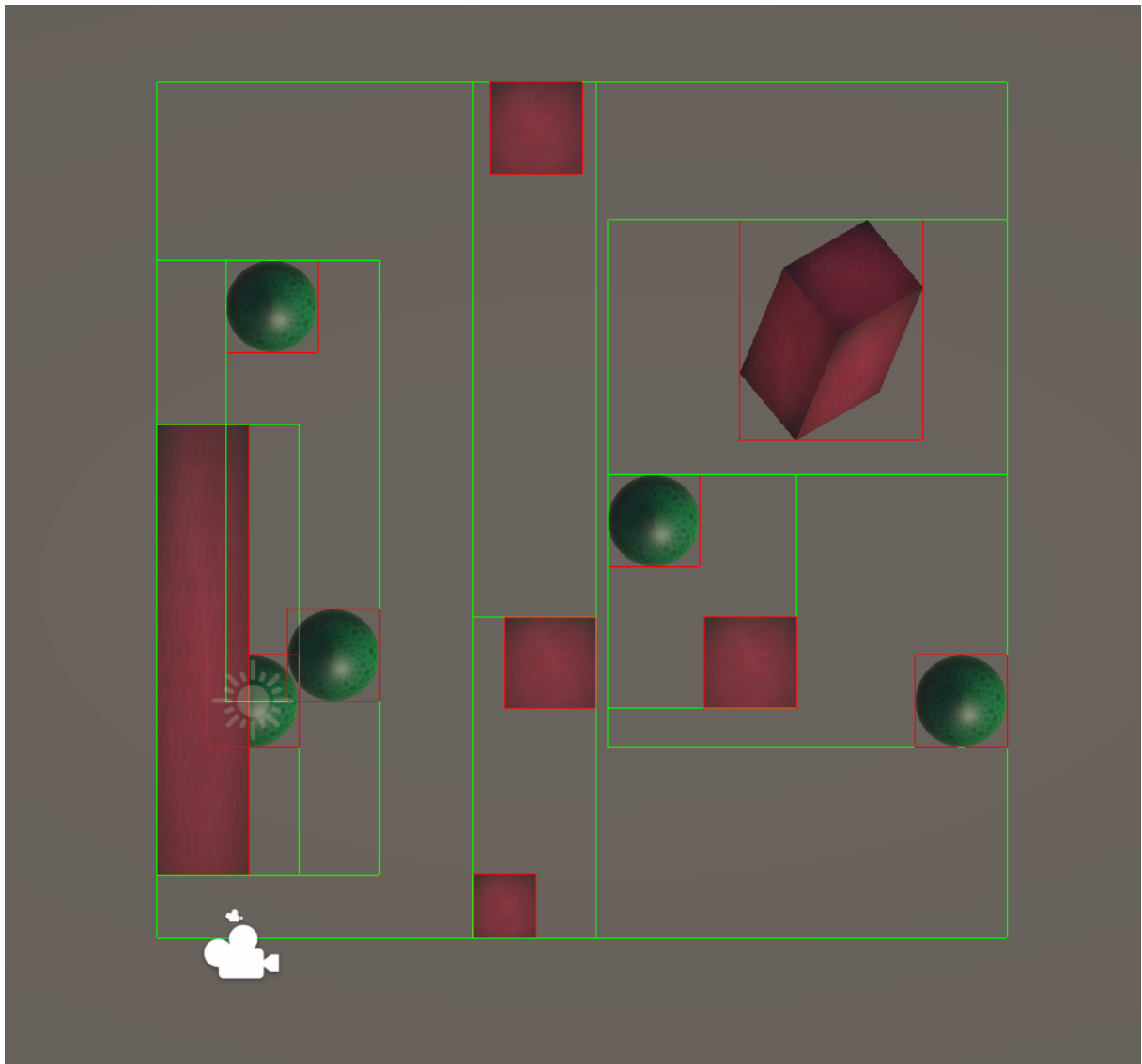


FIGURE 2 – AABB Binary Tree vu de haut.

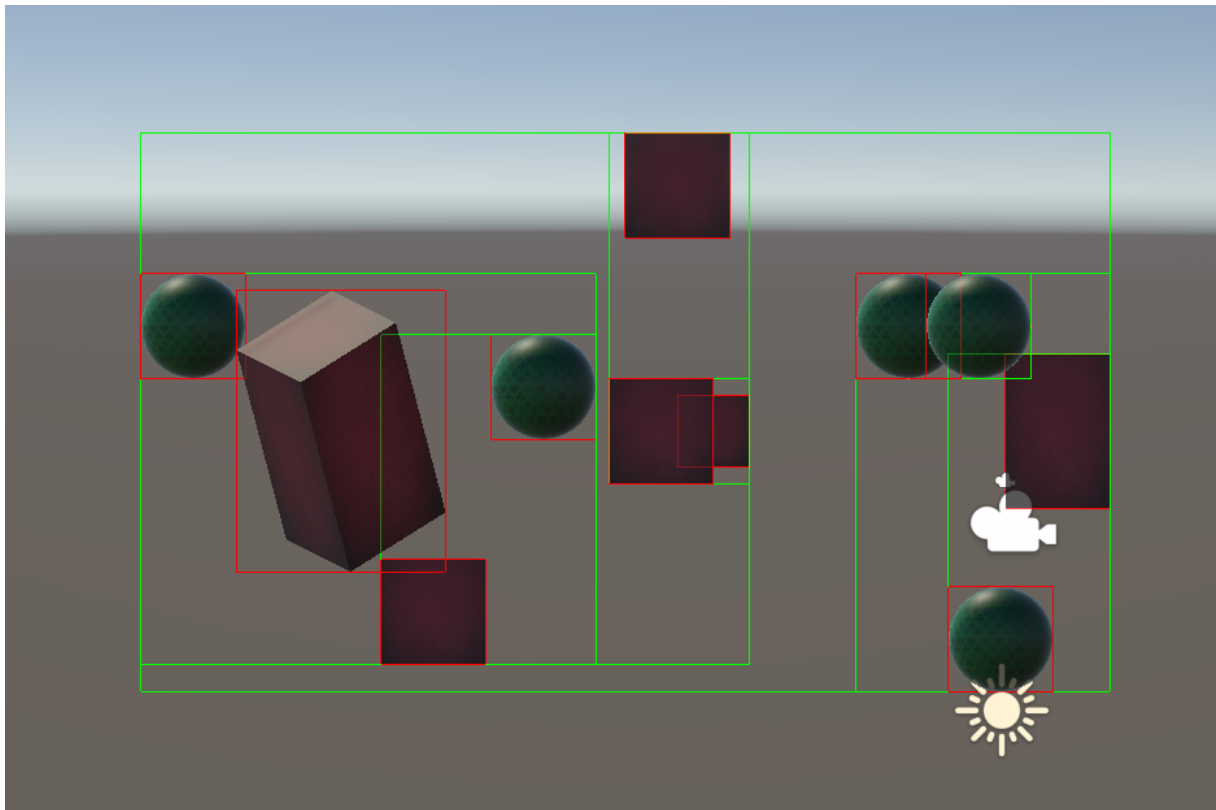


FIGURE 3 – AABB Binary Tree vu de face.

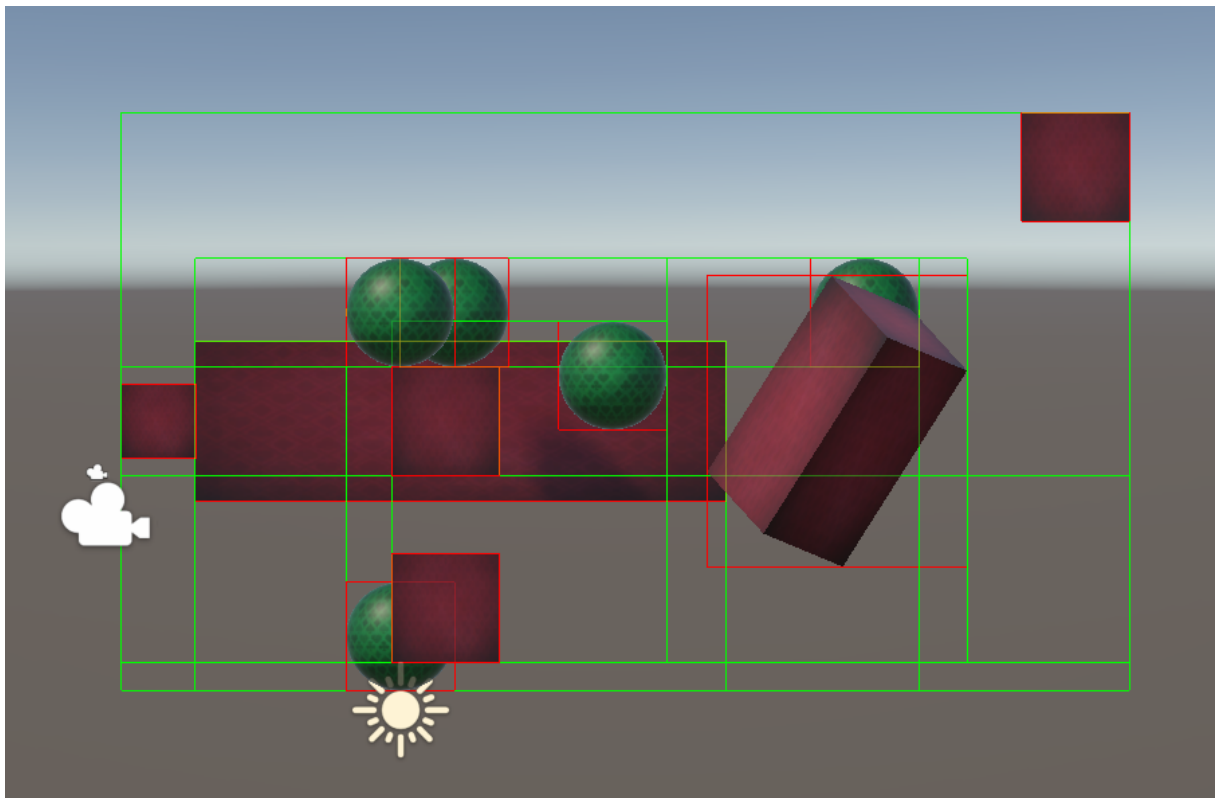


FIGURE 4 – AABB Binary Tree vu de profil.

3 Narrow Phase

La **Narrow Phase** vise à déterminer avec précision si les objets potentiellement en collision se touchent réellement, et à extraire les informations géométriques nécessaires pour la réponse physique (point de contact, normale et profondeur de pénétration).

3.1 Algorithme GJK

Nous utilisons l'algorithme **GJK** (*Gilbert-Johnson-Keerthi*) pour tester l'intersection entre deux volumes convexes. Le principe repose sur la **différence de Minkowski** :

$$M = A \ominus B = \{a - b \mid a \in A, b \in B\}$$

Si l'origine $(0, 0, 0)$ se trouve à l'intérieur de M , les deux volumes sont en collision.

L'algorithme procède de manière itérative :

1. On choisit une direction initiale entre les deux centres.
2. On calcule un point de support dans cette direction :

$$S(\vec{d}) = \text{Support}_A(\vec{d}) - \text{Support}_B(-\vec{d})$$

3. On ajoute ce point au *simplex* et on met à jour la direction vers l'origine.
4. Si le simplex contient l'origine, il y a collision.

Le code implémente les cas 1D, 2D et 3D (segment, triangle, tétraèdre) pour gérer l'évolution du simplex jusqu'à la convergence :

```
while (iterations < max):
    p = GetSupport(colliderA, colliderB, dir)
    if dot(p, dir) <= epsilon: return false
    simplex.add(p)
    if ContainsOrigin(simplex, dir): return true
```

3.2 Algorithme EPA

Lorsque GJK détecte une intersection, nous utilisons **EPA** (*Expanding Polytope Algorithm*) pour déterminer la profondeur et la normale de contact. À partir du simplex final de GJK, EPA agrandit le polytope jusqu'à ce que la face la plus proche de l'origine soit trouvée :

```
for i in range(maxIterations):
    f = getClosestFace(polytope)
    n = f.normal
    s = GetSupport(colliderA, colliderB, n)
    if (dot(n, s) - dot(n, f.a)) <= tolerance:
        return Collision(n, penetration, contactPoint)
    expandPolytope(polytope, s)
```

Cette méthode permet d'obtenir une estimation précise du **vecteur de pénétration**, de la **normale de collision** et du **point de contact moyen**, nécessaires à la résolution des collisions.

4 Corps rigides personnalisés (CustomRigidbody)

La classe `CustomRigidbody` gère désormais :

- le **type** du corps (Static, Kinematic, Dynamic),
- la **masse**, le **centre de masse**, la **vitesse linéaire** et la **vitesse angulaire**,
- le **tenseur d'inertie local** et sa conversion en tenseur d'inertie inverse en espace monde,
- la mise à jour de la position et de l'orientation via une intégration semi-implicite (orientation intégrée à partir de la vitesse angulaire),
- le recalcul automatique des moments d'inertie à partir du collider (boîte ou sphère).

4.1 Résistance aérodynamique

Deux contributions sont modélisées :

Traînée linéaire (drag) La force de traînée appliquée à la translation est calculée par la formule classique :

$$F_{\text{drag}} = -\frac{1}{2} \rho C_d A v^2 \hat{v}$$

avec ρ la densité de l'air, C_d le coefficient de traînée (fourni par le collider) et A la surface de section (fonction de la direction). L'accélération résultante est convertie en modification de vitesse dans le `FixedUpdate`.

Traînée rotationnelle Pour la rotation, un modèle de traînée rotationnelle est appliqué sous forme de couple approximatif proportionnel à ω^2 :

$$\tau_{\text{rot}} \approx -\frac{1}{2} \rho C_{\text{rot}} A r^2 \omega \hat{\omega}$$

où C_{rot} est un coefficient rotationnel (fourni par le collider), r une échelle caractéristique, et ω la vitesse angulaire. Le changement angulaire est obtenu via $\Delta\omega = I^{-1} \tau \Delta t$ (où I^{-1} est le tenseur d'inertie inverse en espace monde).

Amortissement angulaire et plafonnement Un amortissement exponentiel est appliqué à la vitesse angulaire pour éviter les oscillations numériques, et une vitesse angulaire maximale est imposée.

5 Réponse aux collisions

La résolution des collisions combine désormais les contributions linéaires et angulaires, ainsi que les effets de friction et de résistance au roulement.

5.1 Formulation du contact

Pour chaque contact, on dispose :

$$\mathbf{r}_a = \mathbf{p} - \mathbf{c}_1, \quad \mathbf{r}_b = \mathbf{p} - \mathbf{c}_2$$

et de la vitesse relative :

$$\mathbf{v}_{\text{rel}} = (\mathbf{v}_2 + \boldsymbol{\omega}_2 \times \mathbf{r}_b) - (\mathbf{v}_1 + \boldsymbol{\omega}_1 \times \mathbf{r}_a)$$

dont on extrait la composante normale $v_n = \mathbf{v}_{\text{rel}} \cdot \mathbf{n}$.

5.2 Impulsion normale (collision élastique/inélastique)

On calcule l'impulsion de restitution :

$$j = - \frac{(1 + e)v_n}{w_1 + w_2 + \mathbf{n} \cdot \left((I_1^{-1}(\mathbf{r}_a \times \mathbf{n})) \times \mathbf{r}_a + (I_2^{-1}(\mathbf{r}_b \times \mathbf{n})) \times \mathbf{r}_b \right)}$$

puis on applique :

$$\begin{cases} \mathbf{v}_1 \leftarrow \mathbf{v}_1 - w_1 j \mathbf{n} \\ \mathbf{v}_2 \leftarrow \mathbf{v}_2 + w_2 j \mathbf{n} \end{cases} \quad \begin{cases} \boldsymbol{\omega}_1 \leftarrow \boldsymbol{\omega}_1 + I_1^{-1}(\mathbf{r}_a \times (-j\mathbf{n})) \\ \boldsymbol{\omega}_2 \leftarrow \boldsymbol{\omega}_2 + I_2^{-1}(\mathbf{r}_b \times (j\mathbf{n})) \end{cases}$$

5.3 Friction tangentielle

On calcule le vecteur tangent $\mathbf{t}$ (projection du mouvement relatif hors de la normale) et on évalue une impulsion tangentielle j_t :

$$j_t = - \frac{\mathbf{v}_{\text{rel}} \cdot \mathbf{t}}{w_1 + w_2 + \mathbf{t} \cdot \left((I_1^{-1}(\mathbf{r}_a \times \mathbf{t})) \times \mathbf{r}_a + (I_2^{-1}(\mathbf{r}_b \times \mathbf{t})) \times \mathbf{r}_b \right)}$$

La magnitude est bornée par le critère de Coulomb :

$$|j_t| \leq \mu_s |j|$$

avec transition vers une friction dynamique si la limite statique est dépassée. Les impulsions linéaires et angulaires correspondantes sont ensuite appliquées.

5.4 Résistance au roulement et correction

Une résistance au roulement, proportionnelle à ω^2 , freine progressivement la rotation :

$$\tau_{\text{roll}} = -k \mu_d \omega^2 \hat{\omega}$$

et une correction positionnelle proportionnelle à la profondeur de pénétration est appliquée :

$$\Delta \mathbf{p} = \frac{\text{penetration}}{w_1 + w_2} \alpha \mathbf{n}.$$

Des seuils minimaux sur les vitesses linéaires et angulaires éliminent les oscillations numériques résiduelles.

5.5 Résultats

1. Cubes



FIGURE 5 – Cubes avant et après une collision, de masses 15.21kg pour le cube du bas et 10kg pour le cube du haut, avec un coefficient de restitution de 0.55.

2. Sphères

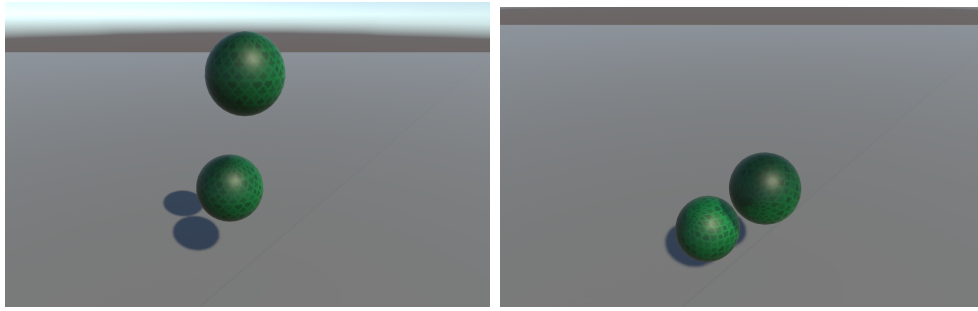


FIGURE 6 – Sphères avant et après une collision, toutes deux de masse 1kg, avec un coefficient de restitution de 1.

3. Correction de la position

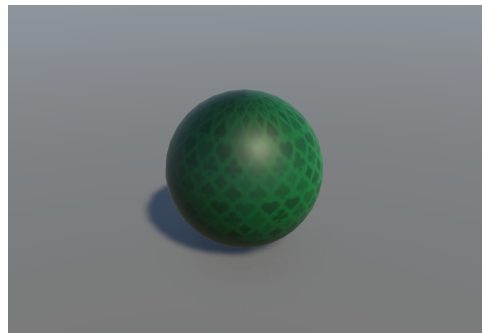


FIGURE 7 – Sphère stabilisée sur le sol grâce à la correction de position.

6 Démonstration

Pour valider notre implémentation, plusieurs scènes ont été réalisées sous *Unity* :

- **Chute libre** : des sphères tombant sur un plan rigide illustrent la gestion du rebond et la cohérence du coefficient de restitution.
- **Empilement de cubes** : test de stabilité et de correction de pénétration dans un système multi-contacts.
- **Collision sphère-boîte** : vérification du fonctionnement combiné de GJK et EPA.
- **Scène complexe** : une centaine d'objets en interaction simultanée démontrant la robustesse du Broad Phase.

Les tests ont montré une simulation stable, avec un comportement réaliste de friction et de rebond, même dans des situations de collisions multiples. La structure AABB Tree permet de maintenir des performances temps réel, avec une complexité proche de $O(n \log n)$.



FIGURE 8 – Jeu de démonstration du moteur physique.

7 Architecture du projet

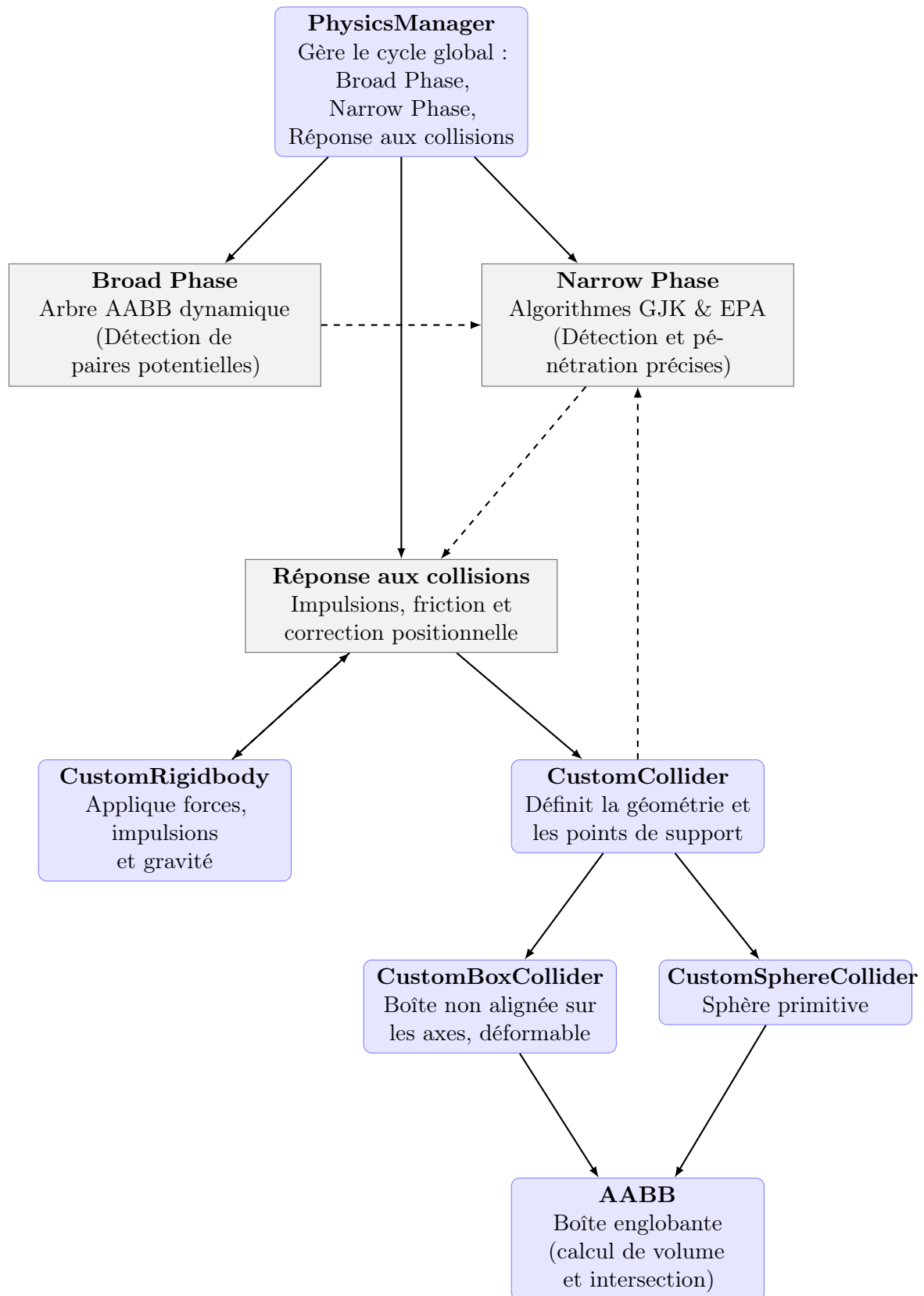


FIGURE 9 – Diagramme de flux de l'architecture du moteur physique 3D.

8 Conclusion

Ce projet a permis de concevoir un **moteur physique 3D complet** et modulaire, intégrant l'ensemble des étapes du pipeline physique moderne : la détection hiérarchique (**Broad Phase**), la détection précise (**Narrow Phase**) et la **réponse dynamique complète** avec impulsions linéaires et angulaires.

Les ajouts récents — notamment la gestion du **tenseur d'inertie**, la **rotation des corps**, les **impulsions angulaires**, la **friction de l'air** (linéaire et rotationnelle) et la **correction positionnelle robuste** — ont permis d'obtenir des comportements réalistes, stables et visuellement cohérents.

Nous avons ainsi approfondi la compréhension :

- des principes mécaniques fondamentaux (inertie, couples, impulsions, frottements),
- des structures de données spatiales (arbres AABB et hiérarchies dynamiques),
- et des algorithmes de géométrie computationnelle (GJK/EPA).

Les perspectives d'évolution incluent :

- l'intégration de contraintes articulées (charnières, ressorts, joints),
- la prise en compte d'autres types de collisioners convexes (capsules, cylindres),
- et l'optimisation parallèle du solveur pour des environnements à grande échelle.

Ce moteur, bien qu'implémenté dans Unity pour le rendu, est entièrement indépendant sur le plan physique et constitue une base solide pour tout développement futur en simulation 3D réaliste.